DEEP LEARNING MINI PROJECT ASSIGNMENT GUIDELINES

Phase 1: Problem Definition & Data Selection

- Choose a narrow scope rather than a massive, production-grade problem. Aim for a well-defined task like example image classification, text sentiment analysis, or simple time-series forecasting.
- Ensure an open-source dataset exists (e.g., from Kaggle or UCI Machine Learning Repository) before finalizing the topic.
- Use manageable dataset sizes that can train quickly on standard hardware.

Phase 2: Coding & Workflow Best Practices

- Do not write the entire project in one giant block of code. Break the pipeline down into clean, reusable Python functions (e.g., separate modules for data ingestion, preprocessing, modeling, and evaluation).
- Use Git and GitHub from day one. This tracks changes and prevents catastrophic data or code loss.
- Build the project using standard libraries like TensorFlow, Keras, or PyTorch. There is no need to code backpropagation from scratch unless specifically requested.

Phase 3: The Deep Learning Pipeline

- Never skip Exploratory Data Analysis (EDA). Check for missing data, normalize pixel values, or tokenize text properly before feeding data to a model.
- Build a simple model (or a classic Machine Learning algorithm like Logistic Regression) to get a baseline metric.
- Do not start with a massive 50-layer network. Start with a basic Multi-Layer Perceptron (MLP) or simple Convolutional Neural Network (CNN), and iterate incrementally.
- Monitor validation metrics closely to ensure the model doesn't overfit the training data. Implement regularization strategies like Dropout or Early Stopping only *after* confirming the basic pipeline works.

Phase 4: Project Evaluation & Presentation

- Do not rely solely on accuracy. Evaluate performance using Precision, Recall, F1-Score, or Confusion Matrices depending on the problem type.
- Plot training vs. validation loss/accuracy curves to visually prove how the model learned.
- **Structure the Project Report**: The final report must follow standard IEEE formats or university rubrics(VTU_CMRIT FORMAT(WILL BE SHARED LATER)). It should cleanly outline:
    - Abstract & Objectives,introduction,literature review
    - Methodology & Architecture Diagram
    - Experimental Results & Graph Comparisons
    - Future Scope & References(IN TOTAL 30 PAGES REPORT)

Timeline (July 22 – October 20, 2026)

Weeks	Phase & Deadlines	Core Tasks & Action Items	
Weeks 1–2	Problem Statement & Data Curation	• Form teams of 2–4 students per batch and update the team mate names and topic selected of your choice in sheet shared(No Existing/Common Projects,No Academic Duplication,Strict GitHub Copying marks deduction of 10 marks).For topic selection literature survey is must	1-Page Project Proposal
	Deadline: Aug 5	• Select domain (Computer Vision, NLP, or Time-Series)	(Problem statement & verified dataset link).
		• Source datasets from Kaggle, UCI, or Hugging Face.	
		• Ensure the dataset has 1,000–5,000 samples minimum.	

Weeks 3–4	EDA & Pipeline Setup	• Setup Google Colab/Kaggle Notebooks with GPU.	Jupyter Notebook/colab
	Deadline: Aug 19	• Clean data (handle missing values, normalize/tokenize).	(Showing complete EDA & ready data loaders).
		• Split data strictly into Train/Val/Test (e.g., 80-10-10).	
Weeks 5–7	Baseline & Iterative Modeling	• Build a simple ML baseline model (e.g., Random Forest).	Working Core Code
	Deadline: Sept 9	• Implement advanced DL/Transfer Learning (ResNet, DistilBERT)	(Comparing baseline vs. deep learning model).

		• Plot Training vs. Validation Loss curves.	
Weeks 8–9	Optimization & Tuning	• Apply regularization (Dropout, Early Stopping).	Saved Model Weights
	Deadline: Sept 23	• Tune hyperparameters (learning rates, optimizers).	(Optimized .h5, .pth, or .onnx files).
		• Calculate Precision, Recall, F1-Score, and Confusion Matrix.	
Weeks 10–12	Deployment & Documentation	• Build a web UI frontend using Streamlit or Gradio.	Final Submission Bundle

	Deadline: Oct 23		(GitHub repo& IEEE Report).
		• Push clean, commented code to GitHub.	
		• Draft the final project report in IEEE format/VTU_CMRIT FORMAT(WILL BE SHARED LATER)	

Mini Project will be evaluated for 100 marks and later will be scaled down to 10 marks,Below you can find weightage of each component

Evaluation Component	Weightage	Compnents
Problem Definition & EDA	15%	Clear objectives, proper data distribution analysis, and preprocessing.

Methodology & Architecture	25%	Correct choice of DL network, proper layer design, and use of GPUs.
Model Optimization	20%	Handling overfitting, tuning hyperparameters, and comparing models.
Evaluation Metrics & UI	20%	Comprehensive metrics (F1-score, Confusion Matrix) and a working Streamlit/Gradio app.
Report & Code Quality	20%	IEEE format report/VTU_CMRIT FORMAT(WILL BE SHARED LATER), clean GitHub repository, and final viva presentation.

Project Guidelines (Mandatory)

1. Project ideas must be unique; common/tutorial-based projects will not be accepted.
2. Reusing major project topics from the current or previous semesters is strictly prohibited.
3. Projects must not be copied from other departments, students, or external sources.
4. Direct implementation or code copied from GitHub is not allowed.
5. All submissions will undergo plagiarism and repository checks.
6. If a project is found to be copied from GitHub or any existing source, **10 marks will be deducted**.